

Advanced-Level

14. Explain the event loop.

It checks the **call stack** → executes tasks → then handles callbacks (callback queue & microtask queue).

15. What are Promises?

Object representing future completion/failure of async task.

16. What is async/await?

Syntactic sugar over Promises. Makes async code look synchronous.

17. What is a closure?

A function that remembers variables from its outer scope, even after that scope is gone.

```
function outer() {  
  let count = 0;  
  return function inner() { count++; return count; }  
}  
let counter = outer();  
console.log(counter()); // 1
```

18. Explain prototype & prototypal inheritance.

Every JS object has a hidden `__proto__` linking it to another object (its prototype). Used for inheritance.

19. What are ES modules (import/export)?

A way to split JS into multiple files.

```
// math.js
export const add = (a,b)=>a+b;
// app.js
import { add } from './math.js';
```

20. What is a constructor in a class?

A special method in class to initialize objects.

```
class Person {
  constructor(name) { this.name = name; }
}
```

21. What are Mutation Observers?

API to watch changes in the DOM (like new nodes added/removed).

22. How does Fetch API work?

Used to make HTTP requests. Returns a Promise.

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(res => res.json())
  .then(data => console.log(data));
```

23. Difference between callbacks and promises?

Callback → function passed into another.

Promise → cleaner async handling, avoids "callback hell".

24. What is the difference between call, apply, and bind?

call → calls function with args.

apply → same but args as array.

bind → returns new function with fixed this.

25. What is hoisting in JavaScript?

Variables (var) and function declarations are moved to the top of their scope before execution.